Introduction to Normalization 📊

Normalization is a process in database design that organizes a relational database's tables to reduce data redundancy and improve data integrity. It aims to eliminate anomalies that can occur during data modification. The process involves decomposing a table into smaller, more specific tables and defining relationships between them.

The primary goal of normalization is to ensure that data is stored logically, preventing inconsistencies that can arise when data is scattered or duplicated across multiple places. The different levels of normalization are called Normal Forms (1NF, 2NF, 3NF, BCNF, etc.).

Data Anomalies

Normalization is crucial because it helps to prevent three types of data anomalies that occur in unnormalized tables. An **anomaly** is an undesirable side effect of an operation (insertion, deletion, or update) that can compromise data integrity.

1. Insertion Anomaly

An **insertion anomaly** occurs when you cannot insert a new row into a table because you don't have the data for all columns, particularly the primary key. This forces you to either insert incomplete data or wait for all data to become available.

- **Example**: Consider a table that stores employee information and the department they work in. If you want to add a new department that has no employees yet, you can't insert a new row because the `EmployeeID` field (which is part of the primary key) would be null or non-existent, violating the primary key constraint.

2. Deletion Anomaly

A **deletion anomaly** occurs when deleting a row from a table inadvertently causes the loss of other, unrelated data. This happens because the data is stored in a single, unnormalized table.

- **Example**: In the same table, if the last employee in a department is deleted, all information about that department (like the department's name and location) is also deleted. This is because the department information is stored with the employee's record.

3. Updation Anomaly

An **updation anomaly** occurs when the same data is stored in multiple places, and an update operation fails to modify all occurrences. This can lead to data inconsistency, where a single piece of information has different values in different records.

- **Example**: If a department's location changes in our unnormalized table, and that department has multiple employees, you would have to update the `DeptLocation` field for every employee in that department. If you forget to update even one record, the data becomes inconsistent. Normalization would place department information in a separate table, requiring only a single update.